

1. Scenario:

You're designing a legal document review agent system. One agent extracts key clauses, another flags risky terms, and the third suggests revisions.

Question:

What kind of agent design pattern fits best, and how would failure handling be incorporated?

- ✧ Use a pipeline workflow with three agents: ClauseExtractor → RiskAnalyzer → RevisionAgent.
- ✧ Each agent processes the previous agent's output step-by-step.
- ✧ Add a validation step after every stage to check output quality.
- ✧ If an agent fails, the system retries the task or sends it back to the previous agent.
- ✧ Maintain logs and fallback rules to ensure reliable document processing.

2. Scenario:

You built an agentic code review platform. Users upload code, and agents offer improvements. But feedback isn't always aligned between agents.

Question:

How do you handle conflicting feedback in a Multi-Agent Review workflow?

- ✧ Introduce a ModeratorAgent or ReviewCoordinator.
- ✧ Collect feedback from all reviewing agents.
- ✧ The moderator compares and analyzes conflicts in suggestions.
- ✧ Apply priority rules or scoring to decide the best recommendation.
- ✧ Generate a final combined review summary for the user.

3. Scenario:

In an educational tutoring system, an agent answers queries, but users often misunderstand explanations.

Question:

How can agent design help personalize and simplify responses for different learners?

- ✧ Add a LearnerProfileAgent to track the student's skill level.
- ✧ The tutor agent adapts explanations based on user understanding level.
- ✧ Use a SimplifierAgent to convert complex answers into easy explanations.
- ✧ Provide examples or step-by-step breakdowns when needed.
- ✧ Continuously update the learner profile from interaction history.

4. Scenario:

You built a Nested Agent system for legal Q&A, but agents sometimes give vague or incomplete answers.

Question:

How do you improve answer depth and reliability using Agentic Design?

- ✧ Add a ResearchAgent to gather supporting legal information.
- ✧ Use a ReasoningAgent to analyze and expand the explanation.
- ✧ Implement a VerificationAgent to check completeness of the answer.
- ✧ If the answer is vague, the system loops back for deeper analysis.
- ✧ Provide structured responses with examples, references, and explanations.

5. Scenario:

A marketing campaign planner agent suggests content based on trends. But it starts giving outdated ideas.

Question:

What agent-based solution ensures real-time content freshness?

- ✧ Integrate a TrendMonitoringAgent connected to live data sources.
- ✧ Fetch current trends from APIs or news feeds.
- ✧ Send the updated trend data to the ContentPlannerAgent.
- ✧ Periodically refresh the knowledge base.
- ✧ Ensure marketing suggestions are generated using the latest data.

6. Scenario:

A project management bot has agents for deadlines, resource allocation, and risk prediction. They function independently but feel disjointed.

Question:

How do you align them without centralizing everything?

- ✧ Introduce a CoordinationAgent that synchronizes agent outputs.
- ✧ Each agent shares its results through a shared data layer.
- ✧ Agents read updates from the shared workspace before acting.
- ✧ The coordinator ensures task dependencies are aligned.
- ✧ This keeps agents coordinated without fully centralizing control.

7. Scenario:

You're designing a secure document generation agent for HR. It must ensure no confidential data is leaked in outputs.

Question:

What agentic design helps enforce strict safety checks?

- ✧ Add a SecurityCheckerAgent before the final output stage.
- ✧ This agent scans responses for confidential or sensitive data.
- ✧ Implement rules to remove or mask private information.
- ✧ Add role-based access control for document generation.
- ✧ Only approved and validated documents are delivered to the user.

8. Scenario:

A language-learning chatbot translates sentences, explains grammar, and gives quizzes. Users complain about inconsistencies.

Question:

How would you fix this using agent architecture?

- ✧ Use a central LanguageContextAgent to manage conversation state.
- ✧ Translation, grammar, and quiz agents share the same context data.
- ✧ Ensure all agents reference the same language rules and examples.
- ✧ The context agent coordinates responses between agents.
- ✧ This maintains consistent explanations and exercises.

9. Scenario:

In a finance app, agents calculate investments and suggest strategies. But they ignore recent news like interest rate hikes.

Question:

What kind of agent integration would solve this?

- ✧ Add a NewsMonitoringAgent connected to financial news APIs.
- ✧ The agent continuously gathers market updates and economic events.
- ✧ Feed this information to the InvestmentStrategyAgent.
- ✧ Update financial models with current market conditions.
- ✧ Provide investment advice that reflects latest economic changes.

10. Scenario:

An e-commerce assistant gives recommendations, tracks orders, and answers queries. But when overloaded, it fails silently.

Question:

How would you improve reliability using agents?

- ✧ Use multiple specialized agents for recommendations, orders, and support.
- ✧ Add a LoadBalancerAgent to distribute requests across agents.
- ✧ Implement health monitoring to detect failing agents.
- ✧ If one agent fails, route the request to a backup agent.
- ✧ Provide fallback responses so the system never fails silently.